



Pontificia Universidad Javeriana
Facultad de Ingenieria
Fundamentos de Ingenieria de Software

Informe de Postmortem

Reporte del ciclo, retrospectiva Starfish y PIP

Milestone 4: Dashboard, Alertas y Cierre del Sistema

Equipo SYNTIX TECH

Profesor	Luis Gabriel Moreno Sandoval
Monitor	Juan Pablo Arias Buitrago
Fecha	19 de mayo de 2026
Repositorio	https://github.com/puj-course/FIS_2610_3517_G4

Documento construido a partir del postmortem original del milestone, sin recortar informacion, adaptado al formato institucional de la plantilla suministrada.

Integrantes

Nombre	Git Hub	Rol Scrum / tecnico
Sebastian Ramirez Maldonado	@Sarm-m	Scrum Master
Sebastian Vargas	@juanvargax	Product Owner / Sprint Planner
Samuel Freile	@samuelfl680	Configuration Manager
Solon Losada	@solonlosada2006	DevOps Engineer
Sebastian Rodriguez Ramirez	@juserora	QA Lead

Tabla 1: Integrantes y roles del equipo SYNTIX TECH.

Tabla de contenido

Integrantes	1
1. Introduccion	4
2. Postmortem – Milestone 4: Dashboard, Alertas y Cierre del Sistema	5
2.1. 1. Resumen Ejecutivo	5
2.2. 2. Contexto de Ingeniería y Negocio	5
2.3. 3. Revisión de Datos del Proceso y Calidad	6
2.3.1. Desempeño real vs. planeado	6
2.3.2. Lecciones aprendidas	6
2.4. 4. Reporte de Roles	6
2.4.1. 4.1 Reporte del Scrum Master – Sebastián Ramírez Maldonado (@Sarm-m)	6
2.4.2. 4.2 Reporte del Product Owner / Sprint Planner – Sebastián Vargas (@juanvargax)	7
2.4.3. 4.3 Reporte del Configuration Manager – Samuel Freile (@samuelff680)	7
2.4.4. 4.4 Reporte del Quality Assurance Lead – Sebastián Rodríguez Ramírez (@Juserora)	7
2.4.5. 4.5 Reporte del DevOps Engineer – Solón Losada (@solonlosada2006) .	7
2.5. 5. Identificación de Causa Raíz	8
2.6. 6. Retrospectiva	8
2.7. 7. Retrospectiva Starfish	9
2.7.1. Seguir haciendo Seguir Haciendo	9
2.7.2. Comenzar a hacer Comenzar a Hacer	9
2.7.3. Dejar de hacer Dejar de Hacer	10
2.7.4. Mas de Más de	10
2.7.5. Menos de Menos de	10
2.8. 8. Conclusiones de Cierre del Proyecto	11
2.9. 9. Respuestas completas a las preguntas guía de la PPT de Postmortem	13
2.9.1. 9.1 Revisión del producto, esfuerzo y proceso	13
2.9.2. 9.2 Evaluación objetiva de roles	14
2.9.3. 9.3 Reporte del ciclo por responsabilidades	16
2.9.4. 9.4 Estrategia de desarrollo y atributos de calidad	16

2.9.5. 9.5 Administración de configuración, riesgos y reutilización 17

2.9.6. 9.6 Reflexión del trabajo en grupo 17

2.9.7. 9.7 Verificación Starfish: respuestas por eje 17

2.9.8. 9.8 PIP - Plan de Mejora del Proceso 18

1. Introduccion

Este informe presenta el postmortem del Milestone 4 del proyecto DriveControl / AutoMinder Enterprise, desarrollado por el equipo SYNTIX TECH en el marco de la asignatura Fundamentos de Ingenieria de Software. El documento conserva de forma integral la informacion del postmortem original y la reorganiza bajo el formato institucional suministrado, manteniendo el contenido de resumen ejecutivo, contexto, revision de calidad, evaluacion de roles, causa raiz, retrospectiva Starfish, plan de mejora y respuestas completas a las preguntas guia de la presentacion de postmortem.

El proposito de este reporte es dejar una evidencia clara del ciclo: que producto se produjo, que esfuerzo se invirtio, que proceso se siguio, que problemas aparecieron, cuales fueron sus causas, que aprendizajes surgieron y que acciones de mejora deben incorporarse en el ciclo siguiente o en proyectos futuros. La estructura facilita la lectura academica y permite defender el trabajo con trazabilidad metodologica, tecnica y de calidad.

2. Postmortem – Milestone 4: Dashboard, Alertas y Cierre del Sistema

Proyecto: Drive Control / Syntix Tech **Stack:** MERN (MongoDB, Express, React, Node.js)
Equipo:

Nombre	Usuario	Rol
Sebastián Ramírez Maldonado	@Sarm-m	Scrum Master
Samuel Freile	@samuelff680	Configuration Manager
Sebastián Rodríguez Ramírez	@Juserora	Quality Assurance Lead
Solón Losada	@solonlosada2006	DevOps Engineer
Sebastián Vargas	@juanvargax	Product Owner / Sprint Planner

2.1. 1. Resumen Ejecutivo

El Milestone 4 constituyó el cierre del ciclo de desarrollo del proyecto Drive Control / Syntix Tech. En este hito se consolidó el producto final listo para producción mediante la implementación de alertas documentales dinámicas conectadas a MongoDB Atlas (SOAT y RTM vencidos), la optimización de la navegación con componentes de enrutamiento nativo ([Link](#)), el refinamiento de la interfaz responsiva, y el despliegue automático mediante un pipeline completo de CI/CD con GitHub Actions, análisis de calidad con SonarCloud y publicación condicional de imágenes en DockerHub. El principal hallazgo crítico del hito fue la detección del Quality Gate de SonarCloud en estado **Failed** por Hotspots de seguridad y baja cobertura de pruebas, lo cual fue remediado mediante una sesión de trabajo asíncrona de alta cohesión del equipo.

2.2. 2. Contexto de Ingeniería y Negocio

El valor de negocio del Milestone 4 fue la entrega de un producto funcional, seguro y desplegable automáticamente. Las alertas dinámicas de SOAT y RTM vencidos representan la funcionalidad de mayor impacto para el usuario final: permiten a los gestores de flota identificar de forma inmediata qué vehículos tienen documentos en riesgo, reduciendo la exposición legal del negocio.

Desde la perspectiva de ingeniería, el logro más significativo fue la consolidación del pipeline de CI/CD como garantía de calidad automatizada: cada cambio en el repositorio pasa por compilación, análisis estático de SonarCloud y, si supera el Quality Gate, publicación condicional de la imagen Docker en DockerHub. Este flujo convierte la calidad en un proceso continuo y no en una actividad puntual al final del desarrollo.

El entorno Docker Compose homogeneizado representa el cierre de la deuda técnica de infraestructura identificada desde el Milestone 2, garantizando que el sistema puede ser ejecutado de forma reproducible en cualquier entorno con un único comando.

2.3. 3. Revisión de Datos del Proceso y Calidad

2.3.1. Desempeño real vs. planeado

- El pipeline de CI/CD fue entregado funcional y completamente automatizado, con publicación condicional de imágenes Docker solo cuando el Quality Gate de SonarCloud es superado.
- SonarCloud detectó el Quality Gate en estado **Failed** en la primera ejecución del pipeline, por Hotspots de seguridad no resueltos y cobertura de pruebas por debajo del umbral mínimo configurado.
- La remediación de los Hotspots fue realizada de forma asíncrona por el equipo, demostrando el nivel más alto de cohesión técnica del proyecto.
- Se detectaron Pull Requests enviados al repositorio sin validación funcional previa en local, confiando exclusivamente en que pasar el pipeline de CI implicaba que la lógica de negocio funcionaba correctamente. Esta práctica generó regresiones que no fueron detectadas por las pruebas automatizadas.
- Los datos demo de prueba causaron inconsistencias en el flujo de alertas cuando no estaban emparejados correctamente con el `OWNER_EMAIL` del usuario en sesión.
- La navegación con componentes **Link** mejoró significativamente la experiencia de usuario al eliminar recargas completas de página entre vistas.

2.3.2. Lecciones aprendidas

- Superar el pipeline de CI es una condición necesaria, pero no suficiente para garantizar que la lógica de negocio funciona correctamente. La validación funcional humana en local sigue siendo irremplazable.
 - La coherencia de los datos demo es un requisito de calidad, no un detalle de configuración: datos sin `OWNER_EMAIL` correcto generan comportamientos impredecibles que pueden confundirse con bugs del sistema.
 - El comando `docker compose down -v` debe ser parte del protocolo estándar de limpieza del entorno de prueba para evitar que volúmenes persistentes defectuosos contaminen los resultados del QA.
 - La cobertura de pruebas debe crecer junto con el sistema, no añadirse como una actividad de cierre de milestone.
-

2.4. 4. Reporte de Roles

2.4.1. 4.1 Reporte del Scrum Master – Sebastián Ramírez Maldonado (@Sarm-m)

El Scrum Master coordinó la respuesta asíncrona del equipo ante el bloqueo del Quality Gate de SonarCloud, logrando la más alta cohesión técnica del proyecto en un momento de alta presión

de entrega. La comunicación fue fluida y las decisiones de remediación fueron tomadas de forma consensuada y rápida. Sin embargo, se detectó que algunos integrantes enviaban Pull Requests sin haber validado los flujos de negocio en local, confiando en el pipeline como único mecanismo de validación. Para futuros proyectos, el Scrum Master debe establecer explícitamente en el Definition of Done que la validación funcional local es un prerequisite para abrir cualquier PR, independientemente del estado del pipeline.

2.4.2. 4.2 Reporte del Product Owner / Sprint Planner – Sebastián Vargas (@juanvargax)

El Product Owner cerró el backlog del proyecto con las funcionalidades de mayor valor para el usuario: alertas dinámicas de SOAT y RTM, navegación fluida con Link y una interfaz responsiva consolidada. Sin embargo, el problema de los datos demo sin OWNER_EMAIL coherente evidenció que las guías de configuración de datos de prueba no fueron definidas como un artefacto formal del sprint. Para futuros proyectos, el Product Owner debe incluir en el sprint backlog una historia de usuario o tarea técnica explícita para la configuración y documentación de los datos de prueba del sistema.

2.4.3. 4.3 Reporte del Configuration Manager – Samuel Freile (@samuelf680)

El Configuration Manager logró en este hito el estándar más alto de gestión de configuración del proyecto: el pipeline de CI/CD con publicación condicional en DockerHub representa una implementación completa del flujo de entrega continua. La estandarización del comando `docker compose down -v` como protocolo de limpieza del entorno fue identificada como una necesidad durante las sesiones de QA y debe formalizarse en la documentación del repositorio como procedimiento estándar. Para futuros proyectos, el Configuration Manager debe incluir esta instrucción en el README desde el inicio del proyecto.

2.4.4. 4.4 Reporte del Quality Assurance Lead – Sebastián Rodríguez Ramírez (@Juserora)

El Milestone 4 fue el hito en que el proceso de QA enfrentó su prueba más exigente: la remediación de Hotspots en SonarCloud bajo presión de cierre del proyecto. El QA Lead coordinó la identificación y priorización de los Hotspots de seguridad, distinguiendo entre los críticos que bloqueaban el Quality Gate y los de menor severidad. El hallazgo de PRs enviados sin validación funcional local fue el aprendizaje más importante del rol: el pipeline no reemplaza el criterio humano. Para futuros proyectos, el QA Lead debe definir casos de prueba de extremo a extremo (E2E) como parte del plan de pruebas del sprint, no como una actividad opcional de cierre.

2.4.5. 4.5 Reporte del DevOps Engineer – Solón Losada (@solonlosada2006)

El DevOps Engineer entregó en este milestone el trabajo técnico más complejo del proyecto: el pipeline de GitHub Actions con análisis de SonarCloud y publicación condicional de imágenes en DockerHub. La lógica de publicación condicional (solo se publica si el Quality Gate es superado) fue una decisión de diseño de pipeline que garantiza que DockerHub solo contiene imágenes que

cumplen los estándares de calidad del equipo. La necesidad de purgar volúmenes persistentes con `docker compose down -v` entre ciclos de prueba fue identificada como una práctica de higiene del entorno que debe documentarse y comunicarse al equipo. Para futuros proyectos, el DevOps Engineer debe incluir scripts de limpieza de entorno como parte de los artefactos de infraestructura del repositorio.

2.5. 5. Identificación de Causa Raíz

La causa raíz principal del Milestone 4 fue la **dependencia excesiva de la automatización del pipeline para validar la corrección funcional del sistema**, manifestada en el envío de Pull Requests sin validación local previa de los flujos de negocio. Esta práctica parte de una confusión conceptual importante: el pipeline de CI valida la compilación, el análisis estático y la cobertura de pruebas automatizadas, pero no puede validar si la lógica de negocio es correcta desde la perspectiva del usuario. La validación funcional humana es irremplazable.

Como causa secundaria, la **falta de higiene en el entorno de prueba local** (ausencia del protocolo `docker compose down -v` para purgar volúmenes persistentes defectuosos) generó que datos residuales de sesiones anteriores contaminaran los resultados del QA, dificultando la reproducción y el diagnóstico de algunos comportamientos inconsistentes del sistema. Adicionalmente, los datos demo sin `OWNER_EMAIL` coherente introdujeron inconsistencias en el flujo de alertas que inicialmente fueron confundidas con bugs del sistema.

2.6. 6. Retrospectiva

El Milestone 4 fue el sprint de cierre, y como todo cierre, llegó con una mezcla de satisfacción y aprendizaje. El equipo entregó un producto funcional, desplegable y con un pipeline de calidad automatizada: tres cosas que ninguno de los integrantes hubiera podido hacer solo al inicio del semestre. Eso es, en esencia, lo que representa el crecimiento del equipo a lo largo del proyecto.

El incidente del Quality Gate de SonarCloud en **Failed** fue el momento de mayor tensión del proyecto. Pero también fue el momento que mejor definió al equipo: en lugar de entrar en pánico o buscar atajos, los integrantes se organizaron de forma asíncrona, priorizaron los Hotspots críticos y los corrigieron metódicamente. Esa respuesta ante la adversidad es la lección de ingeniería de software más valiosa del semestre, más que cualquier patrón de diseño o comando de Docker.

La lección sobre los Pull Requests sin validación funcional local es incómoda pero honesta: el equipo confió demasiado en la automatización. Un pipeline que pasa es una señal positiva, no una garantía de corrección. La diferencia entre un ingeniero junior y uno senior está, en parte, en entender esta distinción. El equipo la aprendió de la forma más efectiva: viviendo las consecuencias.

El proyecto cierra con un entorno homogeneizado, un pipeline de CI/CD funcional, una arquitectura desacoplada con patrones GoF, persistencia en nube real y un proceso de QA formalizado. Es un punto de partida sólido para cualquier proyecto de ingeniería de software que venga después.

2.7. 7. Retrospectiva Starfish

Reglas aplicadas: cada frase inicia con un verbo en infinitivo; ninguna frase se repite entre ejes.

2.7.1. Seguir haciendo Seguir Haciendo

#	Frase	Justificación
1	Realizar pruebas con datos reales conectados a MongoDB Atlas para validar flujos de extremo a extremo.	Los datos reales revelan comportamientos que los datos simulados no pueden reproducir, especialmente en el flujo de alertas.
2	Mantener trazabilidad técnica total entre commits, issues, Pull Requests y releases del pipeline.	Permite reconstruir el histórico del desarrollo y justificar cada decisión técnica con evidencia concreta.
3	Implementar publicación condicional en el pipeline: solo publicar en DockerHub si el Quality Gate de SonarCloud es superado.	Garantiza que el repositorio de imágenes Docker solo contiene versiones que cumplen los estándares de calidad del equipo.
4	Reutilizar componentes existentes antes de crear nuevos, verificando el catálogo de componentes del proyecto.	Reduce la deuda técnica, mejora la consistencia visual y acorta el tiempo de desarrollo de nuevas funcionalidades.
5	Cerrar las tareas del sprint de forma progresiva, sin acumular cierres masivos en los últimos días del milestone.	El cierre progresivo permite detectar dependencias no resueltas con tiempo suficiente para remediarlas antes de la entrega.

2.7.2. Comenzar a hacer Comenzar a Hacer

#	Frase	Justificación
1	Estandarizar el uso de <code>docker compose down -v</code> como protocolo obligatorio de limpieza del entorno de prueba entre ciclos de QA.	Los volúmenes persistentes defectuosos contaminan el entorno de prueba y generan comportamientos inconsistentes difíciles de diagnosticar.
2	Ampliar la cobertura de pruebas unitarias de forma incremental durante el desarrollo, no como actividad de cierre.	La cobertura añadida al final del proyecto genera presión de entrega y pruebas de baja calidad escritas bajo apremio.
3	Implementar validación inline de formularios en tiempo real para mejorar la experiencia del usuario antes de la submisión.	La validación en tiempo real reduce los errores de datos que llegan al backend y mejora la usabilidad del sistema.
4	Definir y documentar guías de configuración de datos demo con <code>OWNER_EMAIL</code> coherente antes de iniciar las sesiones de QA.	Los datos demo mal configurados generaron inconsistencias en el flujo de alertas que inicialmente fueron confundidas con bugs del sistema.
5	Incluir pruebas de extremo a extremo (E2E) en el plan de pruebas de cada sprint como criterio de cierre formal.	Las pruebas E2E validan flujos completos del sistema que las pruebas unitarias no pueden cubrir por su naturaleza aislada.
6	Revisar los límites de validación de todos los formularios del sistema antes del cierre de cada milestone.	Los límites no verificados (longitud máxima, formatos requeridos) son una fuente recurrente de bugs en producción.

2.7.3. Dejar de hacer Dejar de Hacer

#	Frase	Justificación
1	Enviar Pull Requests al repositorio sin haber validado los flujos de negocio de forma funcional en el entorno local.	El pipeline de CI valida compilación y análisis estático, pero no puede reemplazar la verificación funcional humana del comportamiento del sistema.
2	Usar datos demo sin coherencia de <code>OWNER_EMAIL</code> para validar flujos que dependen del aislamiento de datos por usuario.	Los datos mal configurados generan comportamientos que simulan bugs inexistentes y desperdician tiempo de diagnóstico del equipo.
3	Ignorar los Hotspots identificados por SonarCloud bajo el argumento de que no afectan la funcionalidad visible del sistema.	Los Hotspots de seguridad son vulnerabilidades potenciales que SonarCloud marca con evidencia técnica; ignorarlos es asumir una deuda de seguridad activa.
4	Cargar información al sistema sin validación previa del formato y los límites definidos en el modelo de datos.	Los datos con formato incorrecto pueden pasar silenciosamente a la base de datos y generar errores difíciles de rastrear en producción.
5	Ignorar el impacto de la navegación con recarga completa de página en la experiencia del usuario final.	La navegación con <code>Link</code> demostró que el cambio a enrutamiento nativo tiene un impacto perceptible en la fluidez del sistema.

2.7.4. Mas de Más de

#	Frase	Justificación
1	Colaborar de forma asíncrona ante bloqueos técnicos de último minuto, especialmente en la fase de cierre del sprint.	La respuesta asíncrona ante el Quality Gate fallido demostró que el equipo puede resolver problemas complejos sin reuniones sincrónicas de emergencia.
2	Navegar entre vistas del sistema mediante componentes <code>Link</code> en lugar de recargas completas de página.	Mejora la experiencia del usuario, reduce el tiempo de navegación y aprovecha el estado de React entre vistas.
3	Mantener coherencia entre las fuentes de datos del sistema y los componentes de presentación que las consumen.	Las inconsistencias entre el modelo de datos y la capa de presentación son la causa más frecuente de bugs visuales en sistemas de gestión.
4	Monitorear el estado del pipeline en tiempo real durante los ciclos de integración de cierre del sprint.	La detección temprana de un Quality Gate fallido permite iniciar la remediación con más tiempo disponible.
5	Ejecutar pruebas de extremo a extremo manuales sobre los flujos críticos del sistema antes de abrir cualquier PR de cierre.	Los flujos críticos (alertas, autenticación, asignación Conductor-Vehículo) deben verificarse funcionalmente en cada ciclo de integración.

2.7.5. Menos de Menos de

#	Frase	Justificación
1	Usar el botón de navegación del navegador como mecanismo principal de regreso entre vistas del sistema.	Genera recargas completas que interrumpen el estado de la aplicación; el sistema debe proveer su propio mecanismo de navegación.
2	Depender de datos técnicos internos (IDs de MongoDB, estructuras de colección) en los mensajes de alerta visibles para el usuario.	Las alertas deben comunicar valor de negocio en lenguaje del usuario, no detalles de implementación técnica.

#	Frase	Justificación
3	Generar retrabajo por problemas de integración que hubieran podido detectarse con una validación funcional local previa al PR.	Cada retrabajo de integración en la fase de cierre es tiempo que no puede dedicarse a las pruebas E2E del sistema.
4	Mantener una cobertura de pruebas baja que no refleja la complejidad real del sistema entregado.	Una cobertura baja no es solo un problema con SonarCloud: es una señal de que el sistema tiene áreas de riesgo sin verificación automatizada.
5	Generar alertas sin valor claro para el usuario, mostrando información técnica que no le permite tomar decisiones de negocio.	Una alerta efectiva debe responder a la pregunta del usuario: ¿qué debo hacer ahora? No: ¿qué pasó en la base de datos?

2.8. 8. Conclusiones de Cierre del Proyecto

El proyecto Drive Control / Syntix Tech cierra con un sistema funcional, desplegable y mantenible. A lo largo de los cuatro milestones, el equipo recorrió un camino de madurez técnica y metodológica que puede medirse en términos concretos:

- Del código sin criterios de aceptación (M1) a un Quality Gate de SonarCloud activo y remediado (M4).
- De la integración sin validación (M1) a un pipeline de CI/CD completo con publicación condicional (M4).
- De estimaciones optimistas (M1) a Scrum Poker con factor de capacidad real (M3 y M4).
- De un entorno heterogéneo (M2) a un Docker Compose homogeneizado (M4).
- De un QA informal (M1) a reportes estandarizados con pasos de reproducción (M3 y M4).

El entorno homogeneizado con Docker Compose representa la garantía más sólida de la estabilidad final del sistema: cualquier integrante, en cualquier máquina, puede ejecutar **docker compose up** y obtener exactamente el mismo entorno. Ese nivel de reproducibilidad es el estándar de la ingeniería de software profesional.

El mayor aprendizaje del proyecto no es técnico: es que la calidad no es un atributo que se añade al final, sino una disciplina que se practica en cada decisión de cada sprint. Desde cuándo crear el `.env` hasta cómo escribir el mensaje de un commit.

Métrica de cierre	M1	M2	M3	M4
Definition of Done formalizado	NO	Parcial	OK	OK
Pipeline CI/CD activo	NO	NO	Parcial	OK
Entorno homogeneizado (Docker)	NO	NO	NO	OK
QA con reportes estandarizados	NO	NO	OK	OK
Variables de entorno desde Día 1	NO	NO	NO	OK
Scrum Poker en estimaciones	NO	NO	OK	OK
Quality Gate SonarCloud activo	NO	NO	Parcial	OK

2.9. 9. Respuestas completas a las preguntas guía de la PPT de Postmortem

Esta sección se agrega como matriz de verificación para dejar respondidas de forma explícita todas las preguntas guía de la presentación de Postmortem y Retrospectiva Starfish. Las respuestas se formulan con hechos del milestone, evitando evaluar personas y concentrándose en el desempeño del trabajo, el producto, el proceso y las oportunidades de mejora.

2.9.1. 9.1 Revisión del producto, esfuerzo y proceso

Pregunta guía de la PPT	Respuesta aplicada al Milestone 4: Dashboard, Alertas y Cierre del Sistema
¿Qué producto se produjo?	Se produjo el cierre funcional del sistema: alertas dinámicas SOAT/RTM conectadas a MongoDB Atlas, navegación fluida con Link, refinamiento responsivo, pipeline CI/CD con GitHub Actions, SonarCloud y publicación condicional de imágenes Docker.
¿Qué esfuerzo se invirtió para hacerlo?	El esfuerzo se concentró en cierre de backlog, estabilización de alertas, remediación de Quality Gate, validación de datos demo y entrega de infraestructura reproducible.
¿Qué proceso se siguió para hacerlo?	El proceso alcanzó su mayor nivel de automatización con CI/CD, análisis estático y Docker; aun así, se evidenció dependencia excesiva del pipeline para validar lógica funcional.
¿Cómo fue el desempeño real comparado con lo planeado?	El desempeño final fue alto en entrega de producto e infraestructura, pero el primer Quality Gate falló por Hotspots y cobertura baja, lo que obligó a remediación bajo presión.
¿Qué lecciones se aprendieron de esta experiencia?	El equipo aprendió que CI/CD no reemplaza validación funcional local, que la cobertura debe crecer durante el desarrollo y que los datos demo deben ser coherentes con OWNER_EMAIL.
¿Debería usarse un criterio diferente, a nivel individual o de equipo, para el futuro?	Sí. Se debe usar un criterio de cierre final que incluya validación funcional local, Quality Gate aprobado, datos demo consistentes, limpieza de volúmenes y pruebas E2E de flujos críticos.
¿Dónde hay oportunidades para mejorar y por qué?	Las oportunidades están en pruebas E2E, cobertura incremental, validación automática de datos de prueba y documentación de limpieza del entorno.
¿Dónde hubo problemas que deben corregirse para el próximo ciclo?	Se deben corregir PRs sin validación local, baja cobertura, Hotspots no resueltos a tiempo y datos demo inconsistentes.

Pregunta guía de la PPT	Respuesta aplicada al Milestone 4: Dashboard, Alertas y Cierre del Sistema
¿Qué hizo bien el equipo y en qué falló?	Hizo bien: entregar pipeline completo, remediar Quality Gate, consolidar alertas dinámicas y cerrar el producto con infraestructura reproducible. Falló en: confiar demasiado en la automatización, dejar cobertura para cierre y no formalizar datos demo desde el inicio
¿Sirvieron las mejoras propuestas previamente?	Sí. Las mejoras de M3 sobre QA, seguridad y CI/CD sirvieron para detectar y corregir problemas reales. Sin embargo, faltó convertir la validación funcional local en regla obligatoria.
¿Cómo ha sido el desempeño comparado con otros equipos?	No se cuenta con datos verificables de otros equipos, por lo que no se afirma una comparación externa. La comparación válida es interna: frente a todos los hitos anteriores, el M4 fue el más maduro en DevOps y cierre de producto; su brecha principal fue cobertura y validación E2E.
¿Se deben modificar los objetivos y las metas?	Para mantenimiento o proyectos futuros, las metas deben incluir cobertura mínima, E2E, validación funcional local y datos demo formalizados desde el planning.
¿Cómo se deben modificar los procesos?	Hacer obligatoria la validación local antes de PR, crecimiento incremental de pruebas, protocolo <code>docker compose down -v</code> y monitoreo temprano del Quality Gate.
¿Cuáles son las áreas de más alta prioridad para analizar?	Cobertura automatizada, pruebas E2E, validación local, datos demo con <code>OWNER_EMAIL</code> , seguridad SonarCloud y limpieza del entorno Docker.

2.9.2. 9.2 Evaluación objetiva de roles

Rol	¿Qué funcionó?	¿Dónde estuvieron los problemas?	¿Dónde se puede mejorar?	Objetivo de mejoramiento para proyectos futuros o mantenimiento
Scrum Master / Liderazgo	Coordinó respuesta asíncrona al Quality Gate fallido.	No todos validaron funcionalmente antes del PR.	Incluir validación local en DoD.	Ningún PR sin evidencia de prueba funcional local.
Product Owner / Planeación	Cerró funcionalidades de mayor valor: alertas, navegación y responsividad.	Los datos demo no fueron artefacto formal desde el inicio.	Definir guía de datos de prueba.	Asegurar datos demo coherentes para validar valor de negocio.
Configuration Manager / Soporte	Consolidó CI/CD y publicación condicional.	Faltó documentar limpieza de volúmenes desde antes.	Formalizar comandos de higiene del entorno.	Evitar contaminación de QA por volúmenes persistentes.
QA Lead / Calidad	Priorizó Hotspots y hallazgos críticos de cierre.	No había suficientes E2E para cubrir flujos completos.	Definir pruebas E2E obligatorias.	Cubrir alertas, autenticación y asignaciones con pruebas funcionales.
DevOps Engineer / Proceso de entrega	Implementó pipeline con SonarCloud y DockerHub condicional.	La cobertura no creció al ritmo del código.	Hacer que el pipeline exija cobertura incremental.	Publicar solo versiones con calidad comprobada.
Ingenieros / Desarrollo	Entregaron cierre funcional y remediaron problemas críticos.	Algunos PR confiaron en CI sin validar negocio localmente.	Probar flujos críticos antes de enviar PR.	Cerrar cada cambio con evidencia funcional, no solo con pipeline verde.

2.9.3. 9.3 Reporte del ciclo por responsabilidades

Responsabilidad solicitada en la PPT	Respuesta del milestone
Resumen del ciclo	Ciclo de cierre con producto funcional, pipeline completo y aprendizaje fuerte sobre límites de la automatización.
Liderazgo: motivación, compromisos, reuniones y apoyo requerido	La motivación y cohesión fueron altas ante el bloqueo del Quality Gate; se requiere institucionalizar la validación local.
Desarrollo: contenido producido frente a requisitos	El contenido final cumple los requisitos centrales de alertas, navegación y despliegue, aunque la cobertura debe fortalecerse.
Planeación: tareas, seguimiento y compromisos	La planeación logró cerrar backlog, pero debió incluir datos demo y pruebas E2E como tareas explícitas.
Proceso: disciplina de trabajo, medición y seguimiento	El proceso fue automatizado y trazable, pero dependió demasiado de CI para validar funcionamiento.
Calidad: estándares, inspecciones, defectos y PIP	La calidad se evidenció con SonarCloud y remediación; la brecha fue cobertura y pruebas de extremo a extremo.
Soporte: logística, configuración, cambios y riesgos	La logística de Docker y pipeline fue sólida; faltó protocolo documentado temprano de limpieza de volúmenes.
Reporte de ingenieros: planeado vs. ejecutado y mejora personal	Los ingenieros demostraron capacidad de cierre bajo presión, pero deben equilibrar velocidad con evidencia funcional local.

2.9.4. 9.4 Estrategia de desarrollo y atributos de calidad

Pregunta guía	Respuesta
¿La estrategia de desarrollo funcionó?	Funcionó para entregar y desplegar, porque el pipeline dio control. Falló parcialmente al convertirse en sustituto de pruebas funcionales humanas.
¿Qué otros enfoques hubieran sido más adecuados?	Hubiera sido más adecuado escribir pruebas durante cada feature y no al cierre, además de definir datos demo desde planning.
¿Cómo debería cambiarse la estrategia en el futuro?	Para futuros proyectos, todo PR debe incluir evidencia local, pruebas automáticas relevantes y validación de datos.
¿Cómo se tuvo en cuenta la usabilidad?	Mejóro con navegación usando Link , menos recargas y ajustes responsivos.
¿Cómo se tuvo en cuenta la mantenibilidad?	Mejóro por componentes reutilizados, pipeline y trazabilidad; debe complementarse con mayor cobertura.
¿Cómo se tuvo en cuenta la compatibilidad?	Docker Compose mejoró reproducibilidad entre entornos, aunque requiere limpieza de volúmenes para QA consistente.
¿Cómo se tuvo en cuenta el desempeño?	La navegación SPA redujo recargas; no se hizo prueba de carga formal, pero se mejoró fluidez percibida.

Pregunta guía	Respuesta
¿Cómo se tuvo en cuenta la seguridad?	SonarCloud permitió detectar Hotspots y bloquear publicación hasta cumplir Quality Gate.
¿Cómo mejorar estos tópicos en el futuro?	Agregar E2E, cobertura incremental, validación de datos demo, monitoreo de Quality Gate y guía de limpieza Docker.

2.9.5. 9.5 Administración de configuración, riesgos y reutilización

Pregunta guía	Respuesta
¿Funcionó la administración de configuración?	Sí, alcanzó el nivel más alto del proyecto con CI/CD, SonarCloud y publicación condicional.
¿Funcionó el control de cambios?	Funcionó para controlar entregas, pero los PR deben incluir validación local además del pipeline.
¿Cómo mejorarlos?	Documentar <code>docker compose down -v</code> , exigir evidencia local y vincular cada cambio a issue/PR con checklist completo.
¿Fue efectivo el manejo y seguimiento de riesgos?	Mejóro mucho: el Quality Gate actuó como control. Aun así, cobertura y datos demo debieron tratarse como riesgos de cierre.
¿Fue buena la estrategia de reutilización?	Fue buena en componentes y pipeline; debe reforzarse evitando crear nuevos componentes sin revisar existentes.

2.9.6. 9.6 Reflexión del trabajo en grupo

Aspecto de reflexión	Respuesta
Comunicación del grupo	La comunicación asíncrona fue efectiva para remediar el Quality Gate.
Calidad del trabajo de los integrantes	La calidad final fue controlada por pipeline, pero necesita más pruebas E2E.
Planeación de tareas y seguimiento	El cierre fue exitoso, aunque datos demo y cobertura debieron planearse antes.
Mitigación de riesgos	Los riesgos de seguridad se detectaron; los riesgos de datos inconsistentes se manejaron tarde.
Relación con el cliente / stakeholder académico	El stakeholder puede ver valor directo en alertas documentales y producto desplegable.
Relación con las tecnologías	El equipo cerró con dominio mayor de React, MongoDB Atlas, Docker, GitHub Actions y SonarCloud.

2.9.7. 9.7 Verificación Starfish: respuestas por eje

Eje de la estrella	Pregunta de la PPT que responde	Respuesta concreta del milestone
Comenzar a hacer	¿Qué acciones deberíamos comenzar para lograr el éxito, resolver problemas, mitigar riesgos, mejorar calidad, aprender, usar herramientas, distribuir tiempo y comunicar mejor?	Estandarizar limpieza Docker, cobertura incremental, validación inline, datos demo coherentes y E2E.
Más de	¿Qué acciones que ya hacemos deberíamos hacer más para mejorar productividad, comunicación, calidad y aprendizaje?	Hacer más colaboración asíncrona, navegación con <code>Link</code> , monitoreo del pipeline y pruebas manuales de flujos críticos.
Seguir haciendo	¿Qué acciones que ya hacemos bien deberíamos continuar?	Mantener trazabilidad, pruebas con datos reales, publicación condicional y cierre progresivo.
Menos de	¿Qué acciones deberíamos hacer menos porque causan inconvenientes o reducen calidad, comunicación o planeación?	Reducir acumulación de pruebas al final, confianza ciega en CI y uso de datos demo no controlados.
Dejar de hacer	¿Qué acciones deberíamos dejar de hacer porque no son productivas o ya no se necesitan?	Dejar de abrir PR sin validación funcional local, ignorar Hotspots y usar datos sin <code>OWNER_EMAIL</code> coherente.

2.9.8. 9.8 PIP - Plan de Mejora del Proceso

Mejora priorizada	Problema que ataca	Cambio concreto en el proceso	Evidencia esperada
Validación local obligatoria	PRs sin prueba funcional	Añadir evidencia local al checklist de PR	PRs con capturas/comentarios de prueba
Cobertura incremental	Cobertura baja al cierre	Escribir pruebas por feature durante el sprint	Quality Gate estable antes del cierre
Datos demo formales	Inconsistencias por <code>OWNER_EMAIL</code>	Crear guía y semilla controlada de datos	QA reproduce alertas sin falsos bugs
Higiene Docker	Volúmenes contaminan pruebas	Documentar y ejecutar <code>docker compose down -v</code>	Entornos limpios antes de QA